Project 2 – Acme Analytics Linux Platform

Day 7 · Phase 4 starts: where a slow application comes from, stopping a runaway process

Muhammad Bilal Javed · 24 October 2026 · study copy

Project	Acme Analytics – hardened, observable and recoverable Linux platform
Phase	4 – Processes, signals and limits
Hosts worked on	app01
Status	Phase 3 ✅ (Day 6) · Phase 4 🟡 – Step 4.1 concept and architecture done, hands-on run in progress
Author	Muhammad Bilal Javed

> **How to read this report.** 💡 **In plain words** – the idea for any reader. 🔧 **Technical** – the settings and commands. 📸 **Evidence** – the real screenshot. Terms are explained in the glossary. Previous day: Day 6.

---

1. A question from industry – too many backups, and backup logs that grow too big

💡 **In plain words:** a friend who works as a Linux administrator asked: in a real company backups are taken all the time – what happens when the backups, and the backup **logs**, become huge? These are two different problems with two different fixes.

Problem	Industry fix	Already in this project
The backups grow	Retention (e.g. 7 daily, 4 weekly, 12 monthly) · incremental copies · deduplication (restic, borg) · move old copies to cheaper storage (AWS S3 → Glacier with a lifecycle rule)	`find … -mtime +7` keeps 7 days · `rsync --link-dest` hard links (Day 6)
The backup log grows	Log a summary, not every file (`rsync -v` writes one line per file) · rotate and compress logs with `logrotate` · journal size limit (`SystemMaxUse=`) · watch `/backups` and `/var/log` too	One `logger` line per run · logrotate comes in Phase 5

Interview line: *"I keep backups small with incremental copies, retention and cheaper storage for old copies, and I keep logs small by logging summaries and rotating them with logrotate."*

2. Real-world troubleshooting practice – how the incident drills fit in

💡 **In plain words:** to train like an on-call administrator, each phase will end with an **incident drill on my own Acme servers**: something realistic breaks, and I fix it like a real ticket – triage → hypotheses → commands → fix → verify → write-up in GLPI. The scenarios come from a hands-on troubleshooting lab (SadLab) and an L2/L3 incident workbook; only the cases that match this platform are used.

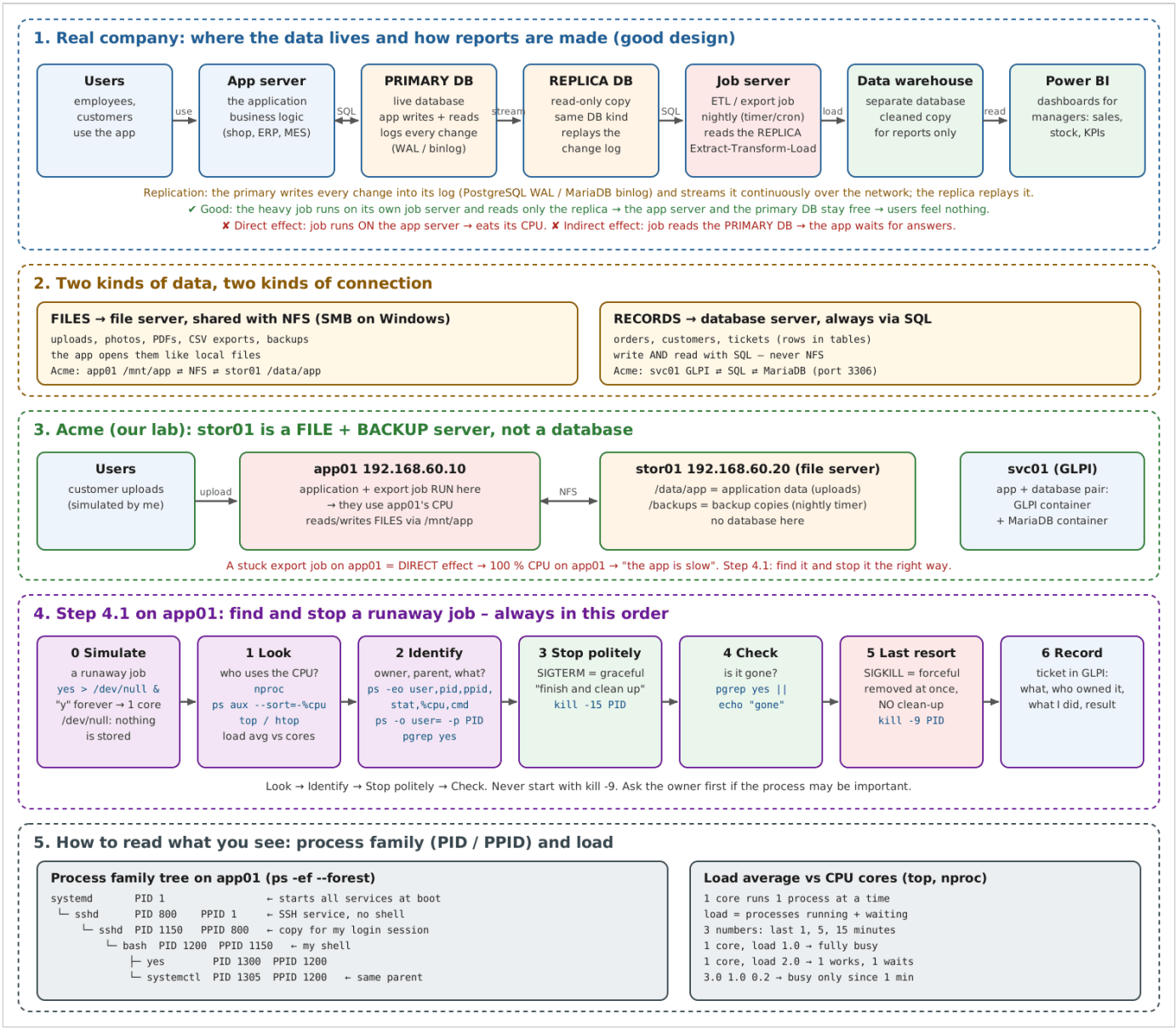
Phase	Drill
4 – Processes, signals	Runaway log writer · deleted file still filling the disk (`lsof +L1`) · "the server is slow"
5 – Logging	Backup log too big → logrotate · log rotation stalls a service
Backups	Backup "succeeds" but the restore fails → add a verify step to the nightly backup
6 – Security	SSH key login rejected · 403 Forbidden · firewall rule blocks a service
7 – Web service	Nginx won't start · works locally, not from outside
8 – Disaster drill	Read-only root after a power cut · wrong device mounted · inodes full

Several "advanced" workbook incidents were already solved for real in this project: a process hung on NFS after a storage failover (Day 5 stale mount and **D state**), an LVM snapshot during maintenance (Day 5), the wrong device mounted (why fstab uses **UUIDs**, Day 4), a backup that must be proven by a restore (Day 4 restore drill), and a cron job versus a timer (Day 6).

3. Phase 4 starts – Step 4.1: where a slow application comes from

💡 **The story:** app01 is Acme's application server. In real companies the most common daily problems are a **process that eats all the CPU**, a **job that hangs**, or a program that **won't stop properly**. Phase 4 teaches how to **find** the guilty process, **stop it the right way**, and finally **cap** what a service may use, so one bad program cannot freeze the whole server. Before touching commands, I worked out where such a process comes from in a real company and in Acme.

📸 **Architecture – where the data lives, how reports are made (primary → replica → job server → data warehouse → Power BI), files vs records, Acme, the order of work on app01, the process family and the load average:**



Step 4.1 architecture

Remember it in four lines:

```
Real company:  Users → App server ⇄ PRIMARY DB → (WAL/binlog stream) → REPLICA → Job server (ETL) → Data warehouse → Power BI
Replication:   the app writes SQL only to the primary; the primary logs every change and streams the log to the replica, which replays it
Acme:          Users → app01 (runs the app) ⇄ NFS (files) ⇄ stor01 (stores files + backups)  ·  svc01: GLPI ⇄ SQL ⇄ MariaDB
Step 4.1:      Simulate → Look → Identify → Stop politely → Check → (kill -9 last) → Record
```

All on **app01**:

```
sudo apt install -y htop stress-ng
nproc                                                    # how many CPU cores
yes > /dev/null &                                        # simulate a runaway job
ps aux --sort=-%cpu | head -5                            # who uses most CPU
top                                                      # load average, %CPU, STAT – q to quit
htop                                                     # same, clearer – q to quit
ps -eo user,pid,ppid,stat,%cpu,cmd --sort=-%cpu | head -5  # owner, parent, state, command
ps -o user= -p $(pgrep yes)                              # who owns it
ps -ef --forest | less                                   # the process family tree
kill -15 $(pgrep yes)                                    # SIGTERM – graceful stop
pgrep yes || echo "gone"                                 # check
```

Term	Meaning
Runaway job	Busy and out of control – 100 % CPU, never finishes
Hung job	Stuck waiting – almost 0 % CPU, never finishes
Load average	Processes running + waiting for the CPU; 1-, 5- and 15-minute averages, compared with `nproc`
PID / PPID	The process's own number / the number of the process that started it
SIGTERM (`kill -15`) / SIGKILL (`kill -9`)	Graceful stop with clean-up / forceful stop, no clean-up – last resort
Primary / replica	Live database / read-only copy of the same kind, kept up to date by streaming the change log
WAL / binlog	The change log of PostgreSQL / MariaDB – the source of replication

📸 **Evidence:** the screenshots of the hands-on run on app01 are added here once the step is done.

Key understanding (in my own words)

1. The problem. The most common complaint about an application server is *"the application is slow"*. Very often the cause is one program using all of the server's CPU. To fix it, I must understand where the data lives, which job can cause the problem, and how to find and stop that job safely.

2. Where the data lives: files and records. Applications use two kinds of data. **Files** (uploads, photos, PDFs, CSV exports, backups) are kept on a **file server** and shared over the network with **NFS** (SMB on Windows); the application opens them as if they were on its own disk. **Records** (orders, customers, tickets) are rows in tables inside a **database**, such as MariaDB or PostgreSQL. The application always talks to a database with **SQL**, both to write ("save this order") and to read ("show me today's orders"), and never with NFS. A server whose main job is running database software is called a **database server**. One database program can hold many separate databases (for example `shop`, `hr` and `glpi`), and two different database programs can even run on the same server, because each uses its own port (MariaDB 3306, PostgreSQL 5432). Most applications come as an **app + database pair**: the app contains the pages, buttons and **business logic** (the rules of what happens, for example "when an order is placed, reduce the stock"), and the database stores the data.

3. How a real company makes reports, and how it goes wrong. Users work in the application (a web shop, ERP or MES) on the **application server**. It writes and reads the live data in the **primary database**, always with SQL and only on the primary. The primary first writes every change into its **change log**, a numbered diary on disk (PostgreSQL: the **WAL**, write-ahead log; MariaDB: the **binlog**), and then changes the table. It streams that log **continuously over the network** to a **replica**, a read-only copy of the **same kind** of database (PostgreSQL → PostgreSQL, MariaDB → MariaDB), which **replays** the same changes on its copy, normally less than a second behind. This is **replication**. Every night an automatic **ETL job** (Extract, Transform, Load), started by a timer or cron, runs on a separate **job server**. It **extracts** millions of rows from the **replica**, **transforms** them (cleans and combines them) and **loads** them into a **data warehouse**, a separate database used only for reports. Moving data between different kinds of database like this is ETL, not replication. **Power BI** reads the data warehouse and shows dashboards to managers, such as sales, stock and KPIs. Typical jobs: *"export today's orders to a CSV file for accounting"*, *"create the monthly sales report"*, *"send new customers to another system every hour"*. This design keeps the heavy job away from the live system, so users feel nothing. If a company skips it, the heavy job slows the application in one of two ways. With a **direct effect**, the job runs **on the application server itself** and uses its CPU. With an **indirect effect**, the job reads the **primary database**, which becomes so busy that the application waits for its answers. A replica protects performance but is **not a backup**: if someone deletes data by mistake, the deletion is copied to the replica within seconds as well.

4. The Acme platform. Acme is smaller. **stor01 is a file and backup server, not a database server**: it holds `/data/app` (the customer uploads, on the LVM volume `lvdata`) and `/backups` (copies made by the nightly backup timer). app01 reads and writes these files through the NFS mount `/mnt/app`. The application and any export job **run on app01**, so a stuck job there is a **direct effect**: it uses app01's CPU, and the application becomes slow. Acme's real app + database pair is GLPI on **svc01**: every ticket, user and inventoried computer is a row in its MariaDB database, and GLPI reads and writes them with SQL.

5. Runaway job, hung job and the load average. A job causes trouble when it is **stuck in a loop**. A loop is a part of a program that repeats the same steps, such as "read the next row, write it, repeat", until it reaches its stop point, for example the last row. Because of a bug, the stop point never comes, so the job repeats forever at 100 % CPU, like a nightly export still running at 10 a.m. That is a **runaway job**: busy and out of control. A **hung job** is different: it uses almost no CPU because it is stuck **waiting**, like my Ctrl+Z waiting on NFS in Phase 3. A normal scheduled job, like my nightly backup, starts, finishes and stops. One **CPU core** runs one process at a time, and `nproc` shows how many cores a server has. The **load average** is the number of processes that want the CPU at the same moment, both running and waiting. `top` shows it as three numbers, the averages over the last **1, 5 and 15 minutes**. With 1 core, a load of 1.0 means fully busy, and 2.0 means one process is working while another waits. `3.0, 1.0, 0.2` means "busy only since a minute ago"; `3.0, 3.0, 3.0` means "busy for at least 15 minutes".

6. Processes, PID and PPID. Every running program is a **process** with its own number, the **PID**. The **PPID** is the PID of the process that **started** it, its parent. Processes form a family tree, shown by `ps -ef --`

`forest`. **systemd** (PID 1) starts all services at boot, so services such as `sshd` have PPID 1 and no shell. When I log in over SSH, sshd starts a copy of itself for my session, and that copy starts my **shell**, bash, the program that reads the commands I type and runs them. Every command I start, for example `yes` or `systemctl restart sshd`, gets its **own PID**, and its **PPID is my bash**. Processes started by the same bash therefore share the same PPID.

7. Finding and stopping the job on app01, always in this order. Simulate → Look → Identify → Stop politely → Check → (kill -9 only as a last resort) → Record. Simulate: I first installed `htop` (a clearer live process viewer) and `stress-ng` (a test tool that creates CPU, memory or disk load on purpose, used later for limits). `yes > /dev/null &` creates a runaway job: `yes` prints the letter “y” without end, so it alone uses one full core; `>` sends this output to `/dev/null`, a special file where everything disappears and nothing is stored (it is not the `/bin` folder, which holds programs); and `&` runs it in the background. **Look:** `nproc` shows the number of cores, and `ps aux --sort=-%cpu | head -5`, `top` and `htop` list the processes using the most CPU. `yes` appears at the top with about 100 %, and the load average rises. **Identify:** `ps -eo user,pid,ppid,stat,%cpu,cmd` shows the **owner** (the user who started it), the PID, the PPID, the state (`R` = running), the CPU use and the **command** (`cmd`, the command line that started it, which shows what the program is). `ps -o user= -p $(pgrep yes)` shows only the owner, and `pgrep yes` shows only the PID. **Stop politely:** `kill -15` sends **SIGTERM**, a **graceful** stop: the program is asked to finish and clean up. **Check:** `pgrep yes || echo "gone"` prints “gone” when nothing is left (`||` means “if the first command finds nothing, run the second”). **Last resort:** `kill -9` sends **SIGKILL**, a **forceful** stop. The kernel removes the process at once and its clean-up never runs, so leftovers such as a lock file or a half-written file can remain; I use it only if a process ignores SIGTERM. **Record:** in a company, before stopping anything, I check who owns the process and whether it is important, asking the owner if needed. Afterwards I write a GLPI ticket: what happened, who owned the process, what I did and the result.

4. Next steps

1. Run Step 4.1 on app01 and add the screenshots; take the VMware snapshot `40-automation` first if not done yet.
2. Steps 4.2–4.5: named pipe, signals and `trap` (why `kill -9` leaves a stale lock file), zombie processes, cgroups (`MemoryMax`, `CPUQuota`) – then the first incident drill.
3. Phase 2 tickets in GLPI.
4. At the end of the project: VMware ESXi / vCenter and Workstation’s *Connect to Server*.